MosaicIMU: Composing Carrier Experts for Generalizable Neural Inertial Odometry

Junye Zou[1,*], **Huiyi Yan**[2,*], **Xinning Xu**[3], **Xiaolei Li**[4], **Pengkun Zhou**[1], **Jinhui Zhang**[5], **Ziyang Meng**[1,†]

[1]State Key Laboratory of Precision Measurement Technology and Instruments, Department of Precision Instrument, Tsinghua University,
[2]Xi'an Jiaotong University, [3]Beijing University of Chemical Technology
[4]Beijing Information Science and Technology University,
[5]Key Laboratory of Complex System Intelligent Control and Decision Making, Beijing Institute of Technology

[*]Equal contribution, [†]Corresponding author

Abstract: Robust inertial odometry is essential for various carriers when external sensing is unreliable. Learning-based methods reduce integration drift by capturing local motion priors, but these methods often remain tied to a particular carrier, limiting generalization across heterogeneous platforms. We present **MosaicIMU**, a carrier-conditioned Mixture-of-Experts (MoE) pretraining-and-adaptation framework for generalizable neural inertial odometry. MosaicIMU uses a prototype-based router to compose carrier-specific expert features, decodes local velocity and uncertainty constraints, and integrates them with a history-aware EKF. For unseen domain adaptation, it freezes the pretrained base model and learns a new lightweight expert residual branch. For edge-deployment, it further reuses the router to select informative online samples for efficient incremental updates. Experiments show that MosaicIMU outperforms the strongest learning-based baseline, reducing average ATE and RTE-10s by 40% and 34%, respectively. These results highlight that MosaicIMU provides a scalable pretraining-to-deployment paradigm for generalizable and adaptive neural inertial odometry.

Keywords: Inertial Odometry, Mixture of Experts, Cross-Carrier Generalization, Online Adaptation

Code: https://github.com/zoujy02/MosaicIMU

1 Introduction

Inertial Measurement Units (IMUs) are widely used for motion estimation on vehicles, pedestrians, quadruped robots, and drones [1]. They are lightweight, low-cost, and robust in visually degraded scenarios, making them attractive when cameras or LiDARs become unreliable [2]. However, inertial odometry is inherently prone to drift, as small errors from bias, vibration, temperature variation, and sensor noise are continuously accumulated through integration [3].

Learning-based inertial odometry mitigates such drift by learning local motion priors from IMU sequences, which capture carrier-dependent relationships between inertial patterns and local motion estimates. Existing methods estimate displacement, velocity, uncertainty, or motion constraints from inertial windows [4, 5, 6], achieving promising results on vehicles [7, 8], pedestrians [6, 9],

E-mail: zoujy21@mails.tsinghua.edu.cn (Junye Zou); ziyangmeng@mail.tsinghua.edu.cn (Ziyang Meng).

10th Conference on Robot Learning (CoRL 2026), Austin TX, USA.

quadruped robots [10], and drones [11, 12]. However, most models are designed for a specific carrier and motion regime. When transferred to another carrier, their learned priors may fail.

A natural expectation is to construct a pretrained foundation model that can be used for heterogeneous carriers. Recent foundation-style IMU models, like TartanIMU, show that large-scale pretraining can learn shared inertial representations across humans, vehicles, robots, and aerial platforms [13]. However, the existing multi-carrier design usually relies on a shared backbone with separate carrier-specific heads. It treats carrier identity as a rigid one-hot condition, and confines cross-carrier knowledge transfer mostly to the shared backbone. In practice, carrier dynamics are not strictly decoupled: different carriers have their own motion characteristics, but they also share common inertial patterns. This raises a key question: *how can we preserve carrier-specific dynamic priors while allowing sample-adaptive feature composition across carriers?*

To this end, we propose **MosaicIMU**, a carrier-conditioned pretraining-and-adaptation framework for generalizable neural inertial odometry across heterogeneous carriers. The name reflects our view that carrier-dependent motion priors should be represented not by a fixed carrier-specific head, but by a sample-adaptive mosaic of carrier-expert features. Our contributions are summarized as follows:

- We propose MosaicIMU, a carrier-conditioned Mixture-of-Experts (MoE) framework that achieves sample-adaptive, feature-level expert composition, and integrates local velocity constraints with a history-aware EKF for stable state estimation.
- For better performance on unseen domains, such as new IMU mounting conditions, new sensor configurations, and new motion patterns, we freeze the pretrained base model and introduce a new lightweight expert residual branch for efficient adaptation.
- For resource-constrained online deployment, we further develop router-guided sample selection, which maintains a compact online buffer of informative samples for efficient incremental updates.
- Extensive experiments demonstrate strong cross-carrier generalization, reducing the average ATE/RTE-10s by 40%/34% compared with the strongest learning-based baseline. For unseen domains, residual adaptation achieves 1.49 m ATE on the TLIO dataset, while router-guided online adaptation reduces ATE from 42.60 m to 7.60 m on self-collected sequences. These results establish that MosaicIMU provides a scalable path toward generalizable and adaptive neural inertial odometry.

2 Related Work

Learning-based inertial odometry. Classical inertial odometry integrates accelerometer and gyroscope measurements to estimate motion, but small errors from sensor bias, noise, vibration, and gravity compensation rapidly accumulate and cause drift. Learning-based methods alleviate this issue by inferring local motion constraints from IMU sequences. Early works such as IONet [4], RoNIN [9], and MotionTransformer [5] regress relative displacement or velocity from inertial windows using deep sequence models. Subsequent studies improve temporal modeling with recurrent, convolutional, or attention-based architectures [14, 15, 16, 17, 18, 8], and enhance robustness through rotation-equivariant learning, frame-aligned representations, or physics-guided inertial features [19, 12]. These methods demonstrate the effectiveness of learning-based inertial odometry, but most are designed for a specific carrier and motion regime.

Carrier-specific and multi-carrier inertial learning. Learned inertial priors are closely coupled with carrier dynamics. Pedestrian methods exploit human motion regularities [9, 6], vehicle methods often rely on planar or non-holonomic assumptions [7, 20], legged-robot methods focus on contact-rich locomotion and bias estimation [10, 21], and aerial methods target highly dynamic UAV motion [11]. Such assumptions work well within each carrier category, but limit generalization across heterogeneous carriers. Recent foundation-style IMU models, such as TartanIMU, pretrain on large-scale heterogeneous data and use carrier-specific output heads to estimate velocity [13]. We follow

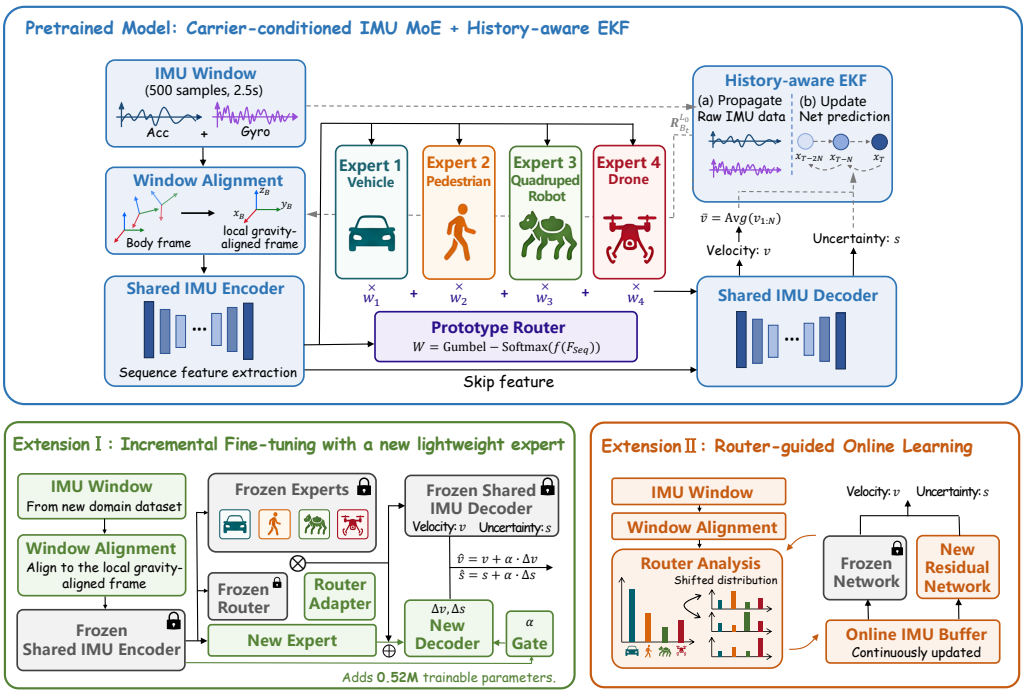


Figure 1: **Overall architecture of MosaicIMU.**

this pipeline but replace the fixed output partition with feature-level adaptive fusion of different carriers, including vehicles, pedestrians, quadruped robots, and drones.

Estimator-compatible learning-based inertial constraints. Another key issue is how learned predictions are represented and fused with a state estimator. Prior works suggest that network outputs should serve as estimator-compatible constraints rather than isolated regression objectives. TLIO fuses learned displacement and uncertainty in a tightly coupled EKF [6], RNIN-VIO incorporates neural inertial estimates into visual-inertial odometry [22], and legged inertial odometry integrates learned IMU displacement into factor-graph estimation [21]. Similarly, we predict dense velocity sequences in a local gravity-aligned frame, avoiding introducing global-yaw observations while maintaining a stable local reference within each window. The predicted sequence is then aggregated into a segment-level velocity observation for a history-aware EKF, improving robustness to frame-wise prediction noise.

3 Method

As shown in Figure 1, MosaicIMU is organized as one pretrained base model and two adaptation extensions. We first train a multi-carrier pretrained MosaicIMU base model, which learns generalizable inertial representations across heterogeneous carriers and integrates the resulting velocity observations with a history-aware EKF. Extension I targets offline adaptation to unseen domains where a target dataset is available. In such a case, we freeze the pretrained base model and update only a lightweight expert residual branch for efficient fine-tuning. Extension II targets online adaptation when target data arrive continuously during online deployment, and the edge computation resource is limited. We then introduce router-guided online sample selection to maintain a compact online buffer of informative samples for efficient incremental updates.

3.1 Carrier-Conditioned Mixture-of-Experts Network

Local gravity-aligned IMU representation. Inspired by TLIO [6], we express each IMU window in a local gravity-aligned frame L_0, whose vertical axis is aligned with gravity and whose yaw

matches the body heading at the start of the window. Let $\mathbf{R}_{B_t}^{W}$ denote the rotation from the instantaneous body frame B_t to the world frame W, whose vertical axis is also aligned with gravity. The initial body yaw defines $\mathbf{R}_{z,0}$, the rotation from L_0 to W, giving $\mathbf{R}_{B_t}^{L_0} = \mathbf{R}_{z,0}^{\top}\mathbf{R}_{B_t}^{W}$. We transform the accelerometer and gyroscope measurements into L_0 and express the ground-truth velocity in the same frame:

$$\mathbf{x}_t^{L_0} = [\mathbf{R}_{B_t}^{L_0}\mathbf{a}_t^{B_t}; \mathbf{R}_{B_t}^{L_0}\boldsymbol{\omega}_t^{B_t}], \qquad \mathbf{v}_t^{L_0} = \mathbf{R}_{z,0}^{\top}\mathbf{v}_t^{W}. \tag{1}$$

This representation provides a fixed reference within each window while remaining invariant to the choice of global yaw reference.

Carrier-conditioned feature fusion. Given the aligned sequence $\mathbf{X}_{1:T}^{L_0} = \{\mathbf{x}_t^{L_0}\}_{1:T}$, a shared encoder E_θ extracts temporal and skip features:

$$\mathbf{F}, \mathbf{u}_1, \mathbf{u}_2 = E_\theta(\mathbf{X}_{1:T}^{L_0}), \tag{2}$$

where $\mathbf{F}$ is the encoded inertial feature, and $\mathbf{u}_1, \mathbf{u}_2$ are multi-scale skip features for decoding. The router g_ϕ maps $\mathbf{F}$ to $\mathbf{z} = g_\phi(\mathbf{F})$, and compares it with K learnable carrier prototypes $\{\mathbf{p}_k\}_{k=1}^{K}$:

$$e_k = \gamma \frac{\mathbf{z}^{\top}\mathbf{p}_k}{\tau\|\mathbf{z}\|\|\mathbf{p}_k\|}, \qquad \{w_k\}_{1:K} = \text{Gumbel-Softmax}([e_1, \ldots, e_K]), \tag{3}$$

where e_k is the similarity logit, w_k is the routing weight, γ is a learnable scale, and τ is the temperature. Gumbel-Softmax is used to encourage the exploration of the feature composition. In this work, K is specified to 4, which represents vehicle, pedestrian, quadruped robot, and drone, respectively.

Under the MoE architecture, each expert G_k produces a carrier-specific feature. Instead of using separate output heads, we use the routing weights to compose expert features at the feature level before a shared decoder D_θ:

$$\mathbf{F}_{moe} = \sum_{k=1}^{K} w_k G_k(\mathbf{F}), \qquad \hat{\mathbf{v}}_{1:T}^{L_0}, \hat{\mathbf{s}}_{1:T} = D_\theta(\mathbf{F}_{moe}, \mathbf{u}_1, \mathbf{u}_2), \tag{4}$$

where $\mathbf{F}_{moe}$ is the fused feature, $\hat{\mathbf{v}}_{1:T}^{L_0}$ is the dense velocity prediction in the local gravity-aligned frame L_0, and $\hat{\mathbf{s}}_{1:T}$ is the predicted velocity variance. This enables sample-adaptive composition of carrier-specific motion priors.

Training objective. We train the base model to predict velocities and their uncertainty, while encouraging distinct carrier prototypes and carrier-aware expert composition. The overall loss function is:

$$\mathcal{L}_{base} = \mathcal{L}_{NLL} + \mathcal{L}_{vel} + \lambda_{orth}\mathcal{L}_{orth} + \lambda_{car}\mathcal{L}_{car}, \tag{5}$$

where λ_{orth} and λ_{car} balance the two regularization terms.

The heteroscedastic Gaussian negative log-likelihood jointly supervises velocity and uncertainty prediction, while an auxiliary squared-error loss provides direct velocity supervision without uncertainty weighting:

$$\mathcal{L}_{NLL} = \frac{1}{T}\sum_{t=1}^{T}\left[\frac{1}{2}(\mathbf{v}_t^{L_0} - \hat{\mathbf{v}}_t)^{\top}\hat{\mathbf{S}}_t^{-1}(\mathbf{v}_t^{L_0} - \hat{\mathbf{v}}_t) + \frac{1}{2}\log|\hat{\mathbf{S}}_t|\right], \quad \mathcal{L}_{vel} = \frac{1}{T}\sum_{t=1}^{T}\|\hat{\mathbf{v}}_t - \mathbf{v}_t^{L_0}\|_2^2, \tag{6}$$

where $\hat{\mathbf{S}}_t = \text{diag}(\hat{\mathbf{s}}_t)$ is the predicted velocity covariance, and $\hat{\mathbf{v}}_t$ is expressed in L_0.

To discourage prototype collapse and promote distinct routing references, we penalize the absolute cosine similarity between carrier prototypes:

$$\mathcal{L}_{orth} = \frac{2}{K(K-1)}\sum_{1\le i<j\le K}\left|\bar{\mathbf{p}}_i^{\top}\bar{\mathbf{p}}_j\right|, \qquad \bar{\mathbf{p}}_i = \frac{\mathbf{p}_i}{\|\mathbf{p}_i\|_2}. \tag{7}$$

To further associate experts with carrier categories while retaining sample-adaptive composition, we regularize the routing distribution $\mathbf{w}$ towards a carrier-specific prior $\mathbf{q}_c = [q_{c,1}, \ldots, q_{c,K}]^{\top}$:

$$\mathcal{L}_{car} = D_{\text{KL}}(\mathbf{w}\|\mathbf{q}_c) = \sum_{k=1}^{K} w_k\left[\log(w_k + \epsilon) - \log(q_{c,k} + \epsilon)\right], \tag{8}$$

where c is the carrier label, $q_{c,k} \geq 0$, $\sum_{k=1}^{K} q_{c,k} = 1$, and ϵ is a small constant for numerical stability.

3.2 Segment-Level Velocity Constraints for History-Aware EKF

Dense velocity predictions provide rich local constraints, but using every frame as an EKF measurement is sensitive to regression noise. We therefore aggregate the dense predictions within each window into a single segment-level average-velocity observation between its two endpoint clones. The filter follows a cloned-pose error-state formulation [23, 24]. Unlike displacement-based neural inertial filters [6, 11], MosaicIMU learns dense local velocities and converts them into velocity-consistency constraints between historical pose pairs.

The filter state is $\mathbf{x} = [\mathbf{R}_B^W, \mathbf{v}^W, \mathbf{p}^W, \mathbf{b}_g, \mathbf{b}_a]$, where $\mathbf{R}_B^W$, $\mathbf{v}^W$, and $\mathbf{p}^W$ are the IMU orientation, velocity, and position in the world frame, and $\mathbf{b}_g, \mathbf{b}_a$ are gyroscope and accelerometer biases. To apply segment-level updates, the filter maintains N historical pose clones, $\mathcal{C} = \{(\mathbf{R}_{B_n}^W, \mathbf{p}_n^W)\}_{n=1}^N$. Raw IMU measurements propagate the current state, whose pose is cloned at regular intervals. Once sufficient history and a complete prediction window are available, each new clone is paired with a historical clone a fixed number of cloning intervals earlier for one update, after which the oldest clone is removed.

For a prediction window associated with clones i and j, let $\mathcal{I}_{ij}$ contain its T prediction frames. We aggregate the network outputs as

$$\mathbf{z}_{ij} = \frac{1}{T} \sum_{t \in \mathcal{I}_{ij}} \hat{\mathbf{v}}_t^{L_i}, \qquad \bar{\mathbf{N}}_{ij} = \operatorname{diag}\left(\frac{1}{T^2} \sum_{t \in \mathcal{I}_{ij}} \hat{\mathbf{s}}_t^{L_i} \right), \tag{9}$$

where $\mathbf{z}_{ij}$ is the neural average-velocity observation and $\bar{\mathbf{N}}_{ij}$ is its approximate diagonal covariance. The filter prediction is the average velocity implied by the cloned poses, expressed in the same local gravity-aligned frame L_i:

$$h_{ij}(\mathcal{C}) = \mathbf{R}_{z,i}^\top \frac{\mathbf{p}_j^W - \mathbf{p}_i^W}{\Delta t_{ij}}, \qquad \mathbf{r}_{ij} = \mathbf{z}_{ij} - h_{ij}(\mathcal{C}), \tag{10}$$

where Δt_{ij} is the time interval between the two clones, $\mathbf{R}_{z,i}$ is the yaw-only rotation from L_i to the world frame, determined by the initial clone's heading, and $\mathbf{r}_{ij}$ is the EKF residual. This residual enforces consistency between the learned local velocity and the historical trajectory without introducing an absolute global-yaw observation. Detailed propagation and update derivations are provided in Appendix A.2.

3.3 Incremental Fine-tuning with a New Lightweight Expert

Although MosaicIMU learns transferable inertial priors, unseen domains may introduce distribution shifts that require adaptation. In Extension I, we adapt the model using available target-domain data by freezing the pretrained base model and training only a router adapter and a gated residual branch.

The router adapter adjusts how pretrained experts are combined for the target domain. Given the encoded feature $\mathbf{F}$, it predicts an additive correction to the frozen base router logits $\boldsymbol{\ell}_{old}$:

$$\Delta\boldsymbol{\ell} = A_\eta(\mathbf{F}), \qquad \mathbf{w}' = \text{Gumbel-Softmax}\left(\boldsymbol{\ell}_{old} + \beta\Delta\boldsymbol{\ell}\right), \tag{11}$$

where $\Delta\boldsymbol{\ell}$ is the correction, $\mathbf{w}'$ is the calibrated routing distribution, and β controls the correction strength. The calibrated prediction is obtained from the frozen experts G_k^{base} and decoder D_θ^{base}:

$$\mathbf{F}_{old} = \sum_{k=1}^{K} w_k' G_k^{base}(\mathbf{F}), \qquad (\hat{\mathbf{v}}_o, \hat{\mathbf{s}}_o) = D_\theta^{base}(\mathbf{F}_{old}, \mathbf{u}_1, \mathbf{u}_2), \tag{12}$$

where $\hat{\mathbf{v}}_o$ and $\hat{\mathbf{s}}_o$ are the base velocity and variance predictions after routing weight calibration.

A lightweight expert G_ψ^{new} and decoder D_ψ^{new} then predict target-domain residuals:

$$\mathbf{F}_{new} = G_\psi^{new}(\mathbf{F}), \qquad \Delta\mathbf{v}, \Delta\mathbf{s} = D_\psi^{new}([\mathbf{F}_{old}, \mathbf{F}_{new}], \mathbf{u}_1, \mathbf{u}_2), \tag{13}$$

where $\Delta\mathbf{v}, \Delta\mathbf{s}$ are velocity and variance residuals. A gate q_ψ controls their magnitude:

$$\alpha = \mathrm{sigmoid}(q_\psi(\mathbf{F})), \qquad \hat{\mathbf{v}} = \hat{\mathbf{v}}_o + \alpha\Delta\mathbf{v}, \quad \hat{\mathbf{s}} = \hat{\mathbf{s}}_o + \alpha\,\mathrm{softplus}(\Delta\mathbf{s}), \tag{14}$$

where α adaptively scales the correction. We use the same loss function as in Eq. (5), except that the term $\mathcal{L}_{orth}$ is omitted.

3.4 Router-Guided Online Adaptation

Unlike Extension I, which utilizes an available offline dataset, Extension II targets real-time online adaptation where IMU windows arrive sequentially and the edge computation resource is limited. We reuse the router as an indicator of carrier-distribution mismatch to reduce the computational burden of online updates. Let $\mathcal{S}_{online} = \{\mathbf{X}_i\}$ denote the recent IMU windows collected from the online stream. For an expected carrier c, the mismatch score m is

$$m(\mathbf{X}) = 1 - \tilde{w}_c(\mathbf{X}), \tag{15}$$

where $\tilde{w}_c(\mathbf{X})$ is the routing probability of carrier c from the calibrated router. We maintain a compact buffer $\mathcal{B}$ by selecting high-mismatch samples:

$$\mathcal{B} \leftarrow \mathrm{TopM}_{\mathbf{X}\in\mathcal{S}_{online}}\, m(\mathbf{X}). \tag{16}$$

Samples in the buffer are used to update the same lightweight components in Section 3.3, while the base MosaicIMU parameters remain frozen.

4 Experimental Results

We evaluate MosaicIMU from three aspects: multi-carrier generalization, lightweight adaptation to unseen domains, and router-guided online learning. We report Absolute Trajectory Error (ATE) for global trajectory accuracy, Relative Trajectory Error over 10 seconds (RTE-10s) for local drift, and z-Drift, defined as ATE computed along the z axis, for vertical positioning accuracy. Implementation details, datasets, and baselines are provided in Appendix B.1 and Appendix B.2.

4.1 Generalization of the Pretrained MosaicIMU

We first compare MosaicIMU with IMO [11], TLIO [6], and TartanIMU [13]. For IMO and TLIO, we train one model per carrier category, yielding four carrier-specific models for each baseline, while TartanIMU and MosaicIMU are trained on the same multi-carrier pretraining datasets. All methods are evaluated on identical held-out trajectories.

As shown in Table 1 and Figure 2, MosaicIMU achieves the best average performance and the lowest ATE across all the baselines. Compared with TartanIMU, which uses fixed carrier-specific heads, our method reduces the average ATE from 11.94 m to 4.15 m and RTE-10s from 3.10 m to 1.31 m. Meanwhile, it also outperforms other learning-based inertial odometry baselines. The gains are especially clear on vehicle and pedestrian sequences, where heterogeneous motion patterns are difficult to capture with a fixed output partition. These results suggest that sample-adaptive feature-level composition provides a more flexible inertial prior than either carrier-specific training or fixed-head multi-carrier training.

Table 2: **Ablation on prediction frames.** Results are averaged over all test carriers. Unit: m.

Prediction Frame	ATE ↓	RTE-10s ↓	z-Drift ↓
Body B_t	8.00	1.93	1.98
Global W	10.40	2.74	1.09
Local-GA L_0	**4.15**	**1.31**	**0.73**

We further ablate the velocity prediction frame in Table 2. Global-frame prediction depends on the recursively estimated attitude, so rotation errors may be injected into the velocity observation and increase trajectory error. Body-frame prediction avoids this global-frame dependence, but the body frame moves and rotates with the carrier each time, making dense velocity predictions less stable. In contrast, the local gravity-aligned frame decouples the velocity observation from global yaw while preserving a gravity-consistent local reference, leading to the lowest ATE, RTE-10s, and z-Drift.

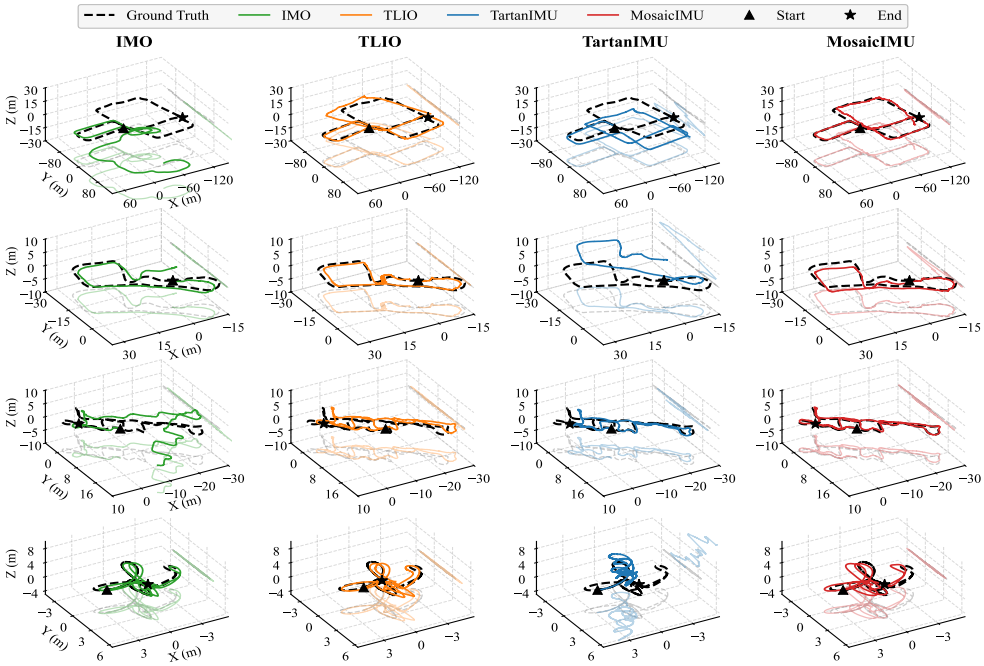


Figure 2: **Comparison of estimated trajectories with ground truth across heterogeneous IMU carriers.** From top to bottom, the four rows correspond to vehicle, quadruped robot, pedestrian, and drone sequences, respectively.

Table 1: **Generalization evaluation on held-out test sequences.** Unit: m.

Carrier	IMO		TLIO		TartanIMU		MosaicIMU	
	ATE ↓	RTE-10s ↓	ATE ↓	RTE-10s ↓	ATE ↓	RTE-10s ↓	ATE ↓	RTE-10s ↓
Vehicle	156.02	13.11	20.87	3.68	25.44	2.77	**10.56** 49%↓	**2.17** 22%↓
Quadruped	3.49	1.22	**1.74**	0.79	3.44	1.17	2.01 16%↑	**0.63** 20%↓
Pedestrian	11.91	6.21	3.83	1.95	10.12	2.39	**2.85** 26%↓	**1.28** 34%↓
Drone	1.29	1.17	1.37	1.48	8.74	6.08	**1.17** 9%↓	**1.14** 3%↓
Average	43.18	5.43	6.95	1.98	11.94	3.10	**4.15** 40%↓	**1.31** 34%↓

Table 3: **Ablation on expert design.** Results are averaged over all test carriers. Unit: m.

Model Variant	ATE ↓	RTE-10s ↓
Single-carrier expert	5.47	2.09
Mixture single expert	5.53	1.74
MosaicIMU	**4.15**	**1.31**

Table 3 further analyzes the design of the expert architecture. The *Single-carrier expert* variant trains four independent single-expert models, one for each carrier, without router-based knowledge sharing. The *Mixture single expert* variant trains one expert using data from all four carriers, also without a router. The former benefits from carrier-specific training but cannot transfer useful priors across carriers, while the latter uses all training data but must represent heterogeneous dynamics with a single shared expert. MosaicIMU achieves the lowest ATE and RTE-10s, showing that multi-carrier inertial learning requires both specialized carrier priors and sample-adaptive feature-level composition. We include detailed ablations in Appendix B.4 and further analyze the router behavior in Appendix B.5.

4.2 Incremental Fine-tuning with a New Lightweight Expert on Unseen Domains

Although multi-carrier pretraining improves generalization, unseen datasets can still introduce sensor noise or motion-distribution shifts. We evaluate adaptation in a head-mounted IMU scenario from the TLIO dataset, which is excluded from pretraining. During fine-tuning, the base MosaicIMU model is frozen, and only the router adapter and gated residual branch are updated. For comparison, we adapt TartanIMU with LoRA [25] under a similar pipeline.

Table 4: **Fine-tuning evaluation on the TLIO dataset.** ATE and RTE-10s are reported at different training epochs. Unit: m.

Method	Trainable Params.	0 Epoch		20 Epoch		40 Epoch		60 Epoch	
		ATE ↓	RTE-10s ↓	ATE ↓	RTE-10s ↓	ATE ↓	RTE-10s ↓	ATE ↓	RTE-10s ↓
TartanIMU + LoRA	0.74M	54.27	16.78	18.82	3.70	12.09	2.53	5.97	2.26
MosaicIMU + New Expert	**0.52M**	**9.85**	**5.36**	**2.60**	**1.32**	**2.22**	**0.88**	**1.49**	**0.72**

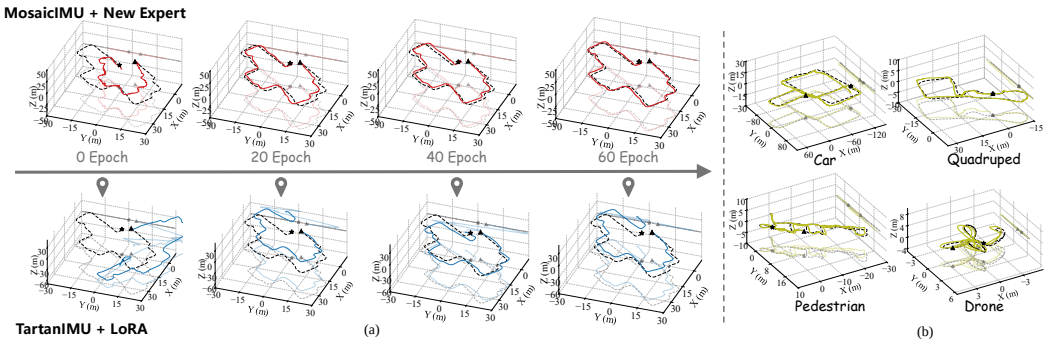


Figure 3: **Fine-tuning adaptation and forgetting analysis.** (a) Trajectory evolution on the unseen TLIO dataset, where MosaicIMU with the new expert residual branch converges faster than TartanIMU with LoRA. (b) Source-domain retention after target-domain fine-tuning, showing that the side-branch design preserves the pretrained multi-carrier knowledge.

Table 4 and Figure 3(a) show the effectiveness of the proposed lightweight adaptation branch. Before fine-tuning, MosaicIMU already achieves lower error than TartanIMU+LoRA, indicating stronger zero-shot generalization from multi-carrier pretraining. With only 0.52M trainable parameters, the adaptation branch reduces ATE to 2.60 m at 20 epochs and 1.49 m at 60 epochs, consistently outperforming TartanIMU+LoRA with fewer updated parameters (0.52M vs. 0.74M). The trajectory evolution in Figure 3(a) further confirms the efficiency of our adaptation mechanism.

Figure 3(b) evaluates source-domain retention after target-domain adaptation. The adapted model still maintains accurate trajectories on the original carrier domains, suggesting that freezing the base MosaicIMU model and learning only a residual branch mitigates catastrophic forgetting. We also conduct a router-adapter ablation, with results reported in Appendix B.6.

4.3 Router-Guided Online Adaptation

We finally evaluate router-guided online adaptation in a self-collected real-world vehicle experiment. A four-wheeled vehicle is operated on campus roads, and the lightweight adaptation branch is updated online on a Jetson AGX Orin. Following the online adaptation pipelines of [26] and [27], we use DLIO [28] to provide velocity supervision, while the network input remains IMU-only. During the warm-up, DLIO also supplies reference observations for filter updates to prevent state divergence from the under-adapted model. After convergence, DLIO is removed, and filter updates use the learned IMU-only velocity constraints. For the reported results below, each epoch checkpoint is evaluated by replaying the collected trajectory using IMU-only inference after online adaptation, with the DLIO estimate used as the reference trajectory for error computation. Further implementation details are provided in Appendix B.7.

Figure 4 shows that trajectory accuracy improves substantially during online adaptation. As the vehicle continues to travel, incoming data are used to progressively update the lightweight adaptation branch. The ATE decreases to 10.29 m after 10 epochs and further to 7.60 m after 40 epochs, while the RTE-10s drops from 10.52 m to 2.12 m. On a held-out test sequence from the same target domain, the adapted model achieves 9.73 m ATE and 1.11 m RTE-10s. This suggests that the selected samples capture transferable distribution shifts rather than merely overfitting to one trajectory.

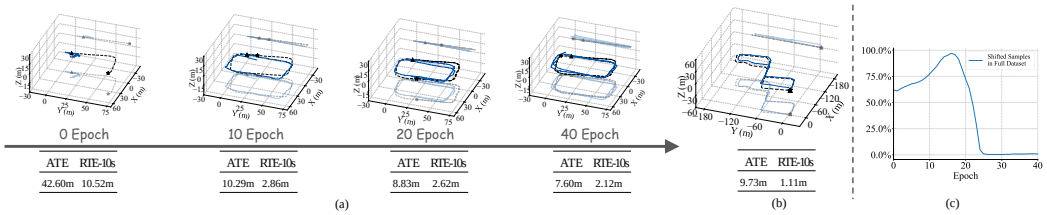


Figure 4: **Router-guided online adaptation.** (a) The online fine-tuning process. (b) Evaluation on a held-out test trajectory not used for online fine-tuning. (c) The proportion of selected high-mismatch samples (shifted samples, weight on vehicle less than 0.8) decreases as the model adapts.

The sample-selection trajectory further illustrates this adaptation behavior. The ratio of shifted samples starts at about 60%, increases briefly, and then drops to 0% around the 25th epoch. This indicates that as the model aligns with the target domain, incoming samples are no longer judged as highly mismatched. Continuing training until 40 epochs further stabilizes the model and improves performance. Here, the router serves not only as an expert-assignment module, but also as a compact signal for selecting informative online samples under edge-computing constraints.

5 Limitations

Despite the improved cross-carrier performance, MosaicIMU remains limited by the current data scale and carrier types, and thus is not expected to handle all possible IMU motion patterns. Larger domain shifts, including new IMU mountings within these categories, can degrade zero-shot performance, which also motivate the lightweight adaptation in Extensions I/II. Meanwhile, broader zero-shot transfer across different hardware or to unseen carrier types has not been systematically evaluated. The filter update also assumes reasonably stable IMU propagation and attitude estimation, so severe bias changes, unusual sensor mounting, and long-term deployment may lead to potential failure cases. In addition, the current online adaptation still requires reference supervision during the early adaptation stage. Future work will broaden the pretraining data, strengthen router uncertainty estimation, and explore more robust self-supervised online adaptation.

6 Conclusion

We present MosaicIMU, a carrier-conditioned MoE framework for generalizable inertial odometry across heterogeneous carriers. Instead of assigning each carrier to a fixed output head, MosaicIMU performs sample-adaptive feature-level expert composition and predicts local velocity constraints for a history-aware EKF. Experiments on four carrier categories show that our method achieves the best average generalization performance, reducing average ATE and RTE-10s by 40% and 34%, respectively, compared with the strongest learning-based baseline. For unseen-domain deployment, the lightweight expert residual adaptation branch further improves the unseen TLIO dataset ATE to 1.49 m, while router-guided online adaptation reduces ATE from 42.60 m to 7.60 m using a compact buffer of selected windows. These results suggest that MosaicIMU offers a practical and scalable step toward general-purpose neural inertial odometry across heterogeneous carriers.

Acknowledgments

This work is supported in part by the opening project from the State Key Laboratory of Autonomous Intelligent Unmanned Systems (No. ZZKF2025ZD-2-2), in part by Tsinghua-Toyota Joint Research Fund, in part by National Natural Science Foundation of China (Grant No. 62633013).

References

[1] N. Ahmad, R. A. R. Ghazilla, N. M. Khairi, and V. Kasi. Reviews on various inertial measurement unit (IMU) sensor applications. *International Journal of Signal Processing Systems*, 1 (2):256–262, 2013.

[2] S. Zhao, H. Zhang, P. Wang, L. Nogueira, and S. Scherer. Super odometry: IMU-centric LiDAR-visual-inertial estimator for challenging environments. In *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 8729–8736. IEEE, 2021.

[3] M. Kok, J. D. Hol, and T. B. Schön. Using inertial sensors for position and orientation estimation. *Foundations and Trends in Signal Processing*, 11(1–2):1–153, 2017.

[4] C. Chen, X. Lu, A. Markham, and N. Trigoni. IONet: Learning to cure the curse of drift in inertial odometry. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, pages 6468–6476, 2018.

[5] C. Chen, Y. Miao, C. X. Lu, L. Xie, P. Blunsom, A. Markham, and N. Trigoni. MotionTransformer: Transferring neural inertial tracking between domains. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 8009–8016, 2019.

[6] W. Liu, D. Caruso, E. Ilg, J. Dong, A. I. Mourikis, K. Daniilidis, V. Kumar, and J. Engel. TLIO: Tight learned inertial odometry. *IEEE Robotics and Automation Letters*, 5(4):5653–5660, 2020.

[7] M. Brossard, A. Barrau, and S. Bonnabel. AI-IMU dead-reckoning. *IEEE Transactions on Intelligent Vehicles*, 5(4):585–595, 2020.

[8] J. Zou, X. Li, P. Gu, X. Lyu, and Z. Meng. MTNet: A multitask learning framework for inertial odometry on vehicles. *IEEE Transactions on Industrial Electronics*, 73(4):5884–5894, 2026.

[9] S. Herath, H. Yan, and Y. Furukawa. RoNIN: Robust neural inertial navigation in the wild: Benchmark, evaluations, and new methods. In *2020 IEEE International Conference on Robotics and Automation (ICRA)*, pages 3146–3152. IEEE, 2020.

[10] R. Buchanan, V. Agrawal, M. Camurri, F. Dellaert, and M. Fallon. Deep IMU bias inference for robust visual-inertial odometry with factor graphs. *IEEE Robotics and Automation Letters*, 8(1):41–48, 2022.

[11] G. Cioffi, L. Bauersfeld, E. Kaufmann, and D. Scaramuzza. Learned inertial odometry for autonomous drone racing. *IEEE Robotics and Automation Letters*, 8(5):2684–2691, 2023.

[12] Y. Qiu, C. Xu, Y. Chen, S. Zhao, J. Geng, and S. Scherer. AirIO: Learning inertial odometry with enhanced IMU feature observability. *IEEE Robotics and Automation Letters*, 10(9):9368–9375, 2025.

[13] S. Zhao, S. Zhou, R. Blanchard, Y. Qiu, W. Wang, and S. Scherer. Tartan IMU: A light foundation model for inertial positioning in robotics. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 22520–22529, 2025.

[14] M. A. Esfahani, H. Wang, K. Wu, and S. Yuan. AbolDeepIO: A novel deep inertial odometry network for autonomous vehicles. *IEEE Transactions on Intelligent Transportation Systems*, 21(5):1941–1950, 2020.

[15] T. Feigl, S. Kram, P. Woller, R. H. Siddiqui, M. Philippsen, and C. Mutschler. A bidirectional LSTM for estimating dynamic human velocities from a single IMU. In *2019 International Conference on Indoor Positioning and Indoor Navigation (IPIN)*, pages 1–8. IEEE, 2019.

[16] R. Li, C. Fu, W. Yi, and X. Yi. Calib-Net: Calibrating the low-cost IMU via deep convolutional neural network. *Frontiers in Robotics and AI*, 8:772583, 2022.

[17] F. Liu, H. Ge, D. Tao, R. Gao, and Z. Zhang. Smartphone-based pedestrian inertial tracking: Dataset, model, and deployment. *IEEE Transactions on Instrumentation and Measurement*, 73:1–13, 2024.

[18] B. Rao, E. Kazemi, Y. Ding, D. M. Shila, F. M. Tucker, and L. Wang. CTIN: Robust contextual transformer network for inertial navigation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 5413–5421, 2022.

[19] R. K. Jayanth, Y. Xu, Z. Wang, E. Chatzipantazis, D. Gehrig, and K. Daniilidis. EqNIO: Subequivariant neural inertial odometry. In *International Conference on Learning Representations (ICLR)*, 2025.

[20] P. Zhou, P. Gu, X. Lyu, M. Liao, and Z. Meng. Learning yaw velocity for inertial-wheel odometry on autonomous vehicles. *IEEE Transactions on Intelligent Vehicles*, 10(5):3517–3530, 2024.

[21] R. Buchanan, M. Camurri, F. Dellaert, and M. Fallon. Learning inertial odometry for dynamic legged robot state estimation. In *Proceedings of the Conference on Robot Learning (CoRL)*, pages 1575–1584. PMLR, 2022.

[22] D. Chen, N. Wang, R. Xu, W. Xie, H. Bao, and G. Zhang. RNIN-VIO: Robust neural inertial navigation aided visual-inertial odometry in challenging scenes. In *2021 IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, pages 275–283. IEEE, 2021.

[23] S. I. Roumeliotis and J. W. Burdick. Stochastic cloning: A generalized framework for processing relative state measurements. In *Proceedings 2002 IEEE International Conference on Robotics and Automation (ICRA)*, volume 2, pages 1788–1795. IEEE, 2002.

[24] A. I. Mourikis and S. I. Roumeliotis. A multi-state constraint Kalman filter for vision-aided inertial navigation. In *Proceedings 2007 IEEE International Conference on Robotics and Automation (ICRA)*, pages 3565–3572. IEEE, 2007.

[25] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

[26] J. Choi, H. Kim, G. Yang, J. Lee, and Y. Cho. KISS-IMU: Self-supervised inertial odometry with motion-balanced learning and uncertainty-aware inference. *arXiv preprint arXiv:2603.06205*, 2026.

[27] Y. Pan, W. Zhou, Y. Cao, and H. Zha. Adaptive VIO: Deep visual-inertial odometry with online continual learning. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 18019–18028. IEEE, 2024.

[28] K. Chen, R. Nemiroff, and B. T. Lopez. Direct LiDAR-inertial odometry: Lightweight LIO with continuous-time motion correction. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 3983–3989. IEEE, 2023.

[29] H. Tang, T. Zhang, L. Wang, X. Ding, M. Yuan, Z. Xiang, J. Chen, Y. Bian, S. Liu, Y. Wang, G. Wang, and X. Niu. i2Nav-Robot: A large-scale indoor-outdoor robot dataset for multi-sensor fusion navigation and mapping. *arXiv preprint arXiv:2508.11485*, 2025.

[30] S. Zhao, Y. Gao, T. Wu, D. Singh, R. Jiang, H. Sun, M. Sarawata, Y. Qiu, W. L. Whittaker, I. Higgins, et al. SubT-MRS dataset: Pushing SLAM towards all-weather environments. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 22647–22657, 2024.

[31] S. Zhu, Z. Xiong, and D. Kim. CEAR: Comprehensive event camera dataset for rapid perception of agile quadruped robots. *IEEE Robotics and Automation Letters*, 9(10):8999–9006, 2024.

[32] S. Yang, Z. Zhang, Z. Fu, and Z. Manchester. Cerberus: Low-drift visual-inertial-leg odometry for agile locomotion. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 4193–4199. IEEE, 2023.

[33] S. Sun, D. Melamed, and K. Kitani. IDOL: Inertial deep orientation-estimation and localization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 6128–6137, 2021.

[34] A. Antonini, W. Guerra, V. Murali, T. Sayre-McCord, and S. Karaman. The Blackbird dataset: A large-scale dataset for UAV perception in aggressive flight. In *International Symposium on Experimental Robotics (ISER)*, pages 130–139. Springer, 2018.

[35] M. A. Shalaby, S. S. Ahmed, N. Dahdah, C. C. Cossette, J. Le Ny, and J. R. Forbes. MILUV: A multi-UAV indoor localization dataset with UWB and vision. *The International Journal of Robotics Research*, page 02783649251405898, 2025.

Appendix

A Method

A.1 Details of Training

All models are implemented in PyTorch and trained on a single NVIDIA RTX 4090 GPU. The IMU data are resampled to 200 Hz and divided into sliding windows of $T = 500$ frames with a stride of 250 frames. Each input window contains 6-D inertial measurements, including three-axis acceleration and angular velocity, and the ground-truth velocity is represented in the local gravity-aligned frame.

For the base model, MosaicIMU is trained on four carrier categories (vehicle, pedestrian, quadruped robot, and drone). We train the network for 100 epochs with a batch size of 128 using the Adam optimizer with a weight decay of 1×10^{-4}. The initial learning rate is set to 1×10^{-3} and decayed to 1×10^{-5} via cosine annealing.

A.2 Details of History-Aware EKF

This section provides the detailed formulation of the history-aware EKF used in Section 3.2. Our estimator builds on the error-state inertial filtering framework with cloned historical poses, which is a standard mechanism for processing delayed relative-state constraints in stochastic cloning and Multi-State Constraint Kalman Filter (MSCKF)-style estimators [23, 24]. Learned inertial odometry systems such as TLIO [6] and IMO [11] also exploit this principle by using network-predicted relative displacement as a filter update. Nevertheless, this work does not regress a single displacement constraint. Instead, the proposed MosaicIMU predicts dense velocity and uncertainty sequences in the local gravity-aligned frame with its carrier-conditioned expert architecture. We design a history-aware velocity-consistency interface that converts dense local predictions into a segment-level average-velocity observation between two cloned poses. This interface preserves the temporal richness of dense velocity regression, suppresses frame-wise prediction noise, and makes the neural output compatible with history-aware state estimation.

State and perturbation convention. We denote the current state as

$$\mathbf{x} = \left(\mathbf{R}_B^W, \mathbf{v}^W, \mathbf{p}^W, \mathbf{b}_g, \mathbf{b}_a\right), \tag{17}$$

where $\mathbf{R}_B^W \in SO(3)$ maps vectors from the body frame B to the world frame W, $\mathbf{v}^W$ and $\mathbf{p}^W$ are the velocity and position in W, and $\mathbf{b}_g, \mathbf{b}_a$ are the gyroscope and accelerometer biases. To support segment-level neural constraints, the filter maintains a bounded set of cloned historical poses,

$$\mathcal{C} = \left\{\left(\mathbf{R}_{B_n}^W, \mathbf{p}_n^W\right)\right\}_{n=1}^N. \tag{18}$$

The augmented state is

$$\mathcal{X} = \left[\mathcal{C}, \mathbf{R}_B^W, \mathbf{v}^W, \mathbf{p}^W, \mathbf{b}_g, \mathbf{b}_a\right]. \tag{19}$$

We use a left-multiplicative perturbation for rotations:

$$\mathbf{R} \leftarrow \mathrm{Exp}(\delta\boldsymbol{\theta})\mathbf{R}, \qquad \mathbf{p} \leftarrow \mathbf{p} + \delta\mathbf{p}. \tag{20}$$

Accordingly, the augmented error state is

$$\delta\mathcal{X} = \left[\delta\boldsymbol{\theta}_1^\top, \delta\mathbf{p}_1^\top, \cdots, \delta\boldsymbol{\theta}_N^\top, \delta\mathbf{p}_N^\top, \delta\boldsymbol{\theta}^\top, \delta\mathbf{v}^\top, \delta\mathbf{p}^\top, \delta\mathbf{b}_g^\top, \delta\mathbf{b}_a^\top\right]^\top, \tag{21}$$

and $\boldsymbol{\Sigma} \in R^{(6N+15)\times(6N+15)}$ denotes the corresponding covariance. Here, N is the number of currently stored clones.

IMU propagation. Let $\tilde{\boldsymbol{\omega}}_k$ and $\tilde{\mathbf{a}}_k$ be the gyroscope and accelerometer measurements at time t_k, and let $\Delta t = t_{k+1} - t_k$. After bias compensation,

$$\boldsymbol{\omega}_k = \tilde{\boldsymbol{\omega}}_k - \mathbf{b}_{g,k}, \qquad \mathbf{a}_k = \tilde{\mathbf{a}}_k - \mathbf{b}_{a,k}. \tag{22}$$

The nominal state is propagated as

$$\begin{aligned} \Delta\boldsymbol{\theta}_k &= \boldsymbol{\omega}_k \Delta t, \\ \mathbf{R}_{k+1} &= \mathbf{R}_k \operatorname{Exp}(\Delta\boldsymbol{\theta}_k), \\ \mathbf{v}_{k+1} &= \mathbf{v}_k + \mathbf{R}_k \mathbf{a}_k \Delta t + \mathbf{g}\Delta t, \\ \mathbf{p}_{k+1} &= \mathbf{p}_k + \mathbf{v}_k \Delta t + \frac{1}{2}\mathbf{R}_k \mathbf{a}_k \Delta t^2 + \frac{1}{2}\mathbf{g}\Delta t^2, \\ \mathbf{b}_{g,k+1} &= \mathbf{b}_{g,k}, \qquad \mathbf{b}_{a,k+1} = \mathbf{b}_{a,k}, \end{aligned} \tag{23}$$

where $\mathbf{g} = [0, 0, -g]^\top$. The biases are kept constant in nominal propagation and modeled as random walks in covariance propagation.

For compactness, define

$$\Delta\mathbf{v}_k = \mathbf{R}_k \mathbf{a}_k \Delta t, \qquad \Delta\mathbf{p}_k = \frac{1}{2}\mathbf{R}_k \mathbf{a}_k \Delta t^2. \tag{24}$$

The current-state error transition matrix $\mathbf{A}_k \in R^{15\times 15}$ is initialized to the identity matrix, with non-zero off-diagonal blocks

$$\begin{aligned} \mathbf{A}_{\mathbf{v}\boldsymbol{\theta}} &= -[\Delta\mathbf{v}_k]_\times, & \mathbf{A}_{\mathbf{p}\boldsymbol{\theta}} &= -[\Delta\mathbf{p}_k]_\times, \\ \mathbf{A}_{\mathbf{pv}} &= \mathbf{I}\Delta t, & \mathbf{A}_{\boldsymbol{\theta}\mathbf{b}_g} &= -\mathbf{R}_{k+1}\mathbf{J}_r(\Delta\boldsymbol{\theta}_k)\Delta t, \\ \mathbf{A}_{\mathbf{v}\mathbf{b}_a} &= -\mathbf{R}_k \Delta t, & \mathbf{A}_{\mathbf{p}\mathbf{b}_a} &= -\frac{1}{2}\mathbf{R}_k \Delta t^2, \end{aligned} \tag{25}$$

where $[\cdot]_\times$ is the skew-symmetric matrix and $\mathbf{J}_r(\cdot)$ is the right Jacobian of $SO(3)$.

The IMU white-noise covariance and bias random-walk covariance are

$$\mathbf{W} = \operatorname{diag}\left(\sigma_g^2 \mathbf{I}_3, \sigma_a^2 \mathbf{I}_3\right), \qquad \mathbf{Q} = \operatorname{diag}\left(\sigma_{b_g}^2 \mathbf{I}_3, \sigma_{b_a}^2 \mathbf{I}_3\right). \tag{26}$$

The IMU noise Jacobian for the current state is

$$\mathbf{B}_k = \begin{bmatrix} \mathbf{R}_{k+1}\mathbf{J}_r(\Delta\boldsymbol{\theta}_k)\Delta t & \mathbf{0} \\ \mathbf{0} & \mathbf{R}_k \Delta t \\ \mathbf{0} & \frac{1}{2}\mathbf{R}_k \Delta t^2 \\ \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{bmatrix}. \tag{27}$$

Pose cloning. Every d IMU samples, the propagated current pose $(\mathbf{R}_{B_c}^W, \mathbf{p}_c^W)$ is copied into the clone set. Each neural observation spans m clone intervals. Once the input buffer is full and $m+1$ clones are available, each newly added clone j is paired with clone $i = j - m$ for one update, after which the oldest clone is removed. Thus, updates occur at every clone insertion after warm-up, with m clones retained between updates.

Since the cloned pose is generated from the current inertial propagation, its uncertainty and cross-correlations with the current state must be inherited from the propagation process. Let $\mathbf{A}_c \in R^{15\times 15}$ denote the error-state transition from the previous IMU state to the clone time. This transition maps the previous current-state error to the propagated 15-D current-state error. However, each clone only stores attitude and position, rather than the full inertial state. Therefore, the clone-augmentation Jacobian is obtained by selecting the attitude and position rows of $\mathbf{A}_c$:

$$\mathbf{J}_c = \begin{bmatrix} \mathbf{A}_{c,\boldsymbol{\theta}} \\ \mathbf{A}_{c,\mathbf{p}} \end{bmatrix} \in R^{6\times 15}. \tag{28}$$

With N existing clones, the augmented transition matrix is

$$\mathbf{A}_k^{aug} = \begin{bmatrix} \mathbf{I}_{6N} & \mathbf{0} \\ \mathbf{0} & \mathbf{J}_c \\ \mathbf{0} & \mathbf{A}_k \end{bmatrix}, \tag{29}$$

and the corresponding augmented IMU noise Jacobian is

$$\mathbf{B}_k^{aug} = \begin{bmatrix} \mathbf{0}_{6N\times 6} \\ \mathbf{B}_c \\ \mathbf{B}_k \end{bmatrix}, \tag{30}$$

where $\mathbf{B}_c$ selects the attitude and position components of the propagation noise Jacobian. If no clone is inserted, $\mathbf{A}_k^{aug}$ reduces to an identity mapping for old clones and $\mathbf{A}_k$ for the current state.

The covariance is propagated by

$$\mathbf{\Sigma}_{k+1} = \mathbf{A}_k^{aug}\mathbf{\Sigma}_k\mathbf{A}_k^{aug\top} + \mathbf{B}_k^{aug}\mathbf{W}\mathbf{B}_k^{aug\top} + \mathbf{G}_b\mathbf{Q}\mathbf{G}_b^{\top}\Delta t, \tag{31}$$

where $\mathbf{G}_b$ selects the bias components of the current state. The propagated covariance preserves cross-correlations between historical poses and the current state.

Filter-aware aggregation of dense neural velocities. For an IMU window spanning the m intervals between clones $i = j - m$ and j, the prediction length is chosen as $T = md$ to match the observation interval. MosaicIMU outputs dense local velocities and the corresponding variances,

$$\left\{ \left(\hat{\mathbf{v}}_t^{L_i}, \hat{\mathbf{s}}_t^{L_i} \right) \right\}_{t=1}^{T}, \tag{32}$$

where L_i denotes the local gravity-aligned frame associated with the start clone i, as defined in Section 3. Its vertical axis is aligned with gravity, while its yaw is anchored by the initial clone. The formulation below assumes that the neural velocities and the cloned-pose prediction use this same local-frame anchor. Let ψ_i be the yaw angle extracted from $\mathbf{R}_{B_i}^{W}$, and define

$$\mathbf{R}_{z,i} = \begin{bmatrix} \cos\psi_i & -\sin\psi_i & 0 \\ \sin\psi_i & \cos\psi_i & 0 \\ 0 & 0 & 1 \end{bmatrix}, \qquad \mathbf{R}_W^{L_i} = \mathbf{R}_{z,i}^{\top}. \tag{33}$$

We would like to remark that such a local gravity-aligned frame has two advantages. First, the measurement in the local gravity-aligned frame avoids adding artificial global-yaw information to the EKF update. In IMU-only estimation, global yaw cannot be directly observed. If the velocity is expressed in the world frame, its direction may be incorrectly treated as a global heading observation by the filter. Second, rotating IMU samples and velocity targets into a gravity-aligned frame provides the network with an implicit gravity reference. This allows MosaicIMU to learn carrier-dependent motion priors in a physically consistent coordinate system, while avoiding the instability of instantaneous body-frame targets.

Rather than treating each instantaneous velocity as an independent measurement, we introduce a segment aggregation operator

$$\mathcal{M}_{ij}\left(\left\{ \hat{\mathbf{v}}_t^{L_i} \right\}_{t\in\mathcal{I}_{ij}} \right) = \sum_{t\in\mathcal{I}_{ij}} \alpha_t \hat{\mathbf{v}}_t^{L_i}, \qquad \sum_{t\in\mathcal{I}_{ij}} \alpha_t = 1, \tag{34}$$

where $\mathcal{I}_{ij}$ contains all T prediction frames in the window associated with clones i and j. For uniformly sampled IMU predictions, we set $\alpha_t = 1/T$, so that each complete prediction window contributes one average-velocity observation. The resulting neural observation is

$$\mathbf{z}_{ij} = \mathcal{M}_{ij}\left(\left\{ \hat{\mathbf{v}}_t^{L_i} \right\}_{t\in\mathcal{I}_{ij}} \right). \tag{35}$$

Neglecting cross-covariances between frame-wise prediction errors, we approximate the observation covariance by aggregating and scaling the predicted variances over the same window:

$$\mathbf{N}_{ij} = \sum_{t\in\mathcal{I}_{ij}} \alpha_t^2 \operatorname{diag}\left(\hat{\mathbf{s}}_t^{L_i} \right). \tag{36}$$

Velocity-consistency measurement model. The filter prediction corresponding to $\mathbf{z}_{ij}$ is derived from the average motion of the two cloned poses. Define the world-frame segment velocity induced by the cloned poses as

$$\bar{\mathbf{v}}_{ij}^{W} = \frac{\mathbf{p}_j^W - \mathbf{p}_i^W}{\Delta t_{ij}}, \qquad \Delta t_{ij} = t_j - t_i. \tag{37}$$

Note that this quantity is not obtained by directly integrating the network prediction, but is a state-dependent velocity implied by the historical trajectory maintained by the filter. Projecting it into the local gravity-aligned frame gives

$$h_{ij}(\mathcal{X}) = \mathbf{R}_W^{L_i} \bar{\mathbf{v}}_{ij}^{W} = \mathbf{R}_{z,i}^{\top} \frac{\mathbf{p}_j^W - \mathbf{p}_i^W}{\Delta t_{ij}}. \tag{38}$$

The residual is therefore

$$\mathbf{r}_{ij} = \mathbf{z}_{ij} - h_{ij}(\mathcal{X}). \tag{39}$$

This residual defines a velocity-consistency constraint between the dense neural velocity field and the historical trajectory estimated by the EKF.

Yaw consistency. Although h_{ij} depends on the initial clone yaw through $R_{z,i}$, this yaw only defines the local gravity-aligned frame of the network observation and does not provide an absolute-yaw measurement. Therefore, for any global yaw rotation R_ψ, applying $p_i^W \leftarrow R_\psi p_i^W$, $p_j^W \leftarrow R_\psi p_j^W$, and $R_{z,i} \leftarrow R_\psi R_{z,i}$ leaves the measurement unchanged:

$$(R_\psi R_{z,i})^{\top} \frac{R_\psi (p_j^W - p_i^W)}{\Delta t_{ij}} = R_{z,i}^{\top} \frac{p_j^W - p_i^W}{\Delta t_{ij}}.$$

This means that the residual is independent of the arbitrary global-yaw reference when the neural observation is expressed in the same local frame. The measurement model therefore enforces consistency between the cloned-pose displacement and the local average velocity without introducing an explicit absolute-heading observation.

Measurement Jacobian. The measurement function depends on the initial clone yaw only through the local-frame anchor, and on the positions of the start and end clones. For compactness, denote

$$\mathbf{q}_{ij} = \bar{\mathbf{v}}_{ij}^{W} = \frac{\mathbf{p}_j^W - \mathbf{p}_i^W}{\Delta t_{ij}}, \qquad \hat{\mathbf{z}}_{ij} = \mathbf{R}_{z,i}^{\top} \mathbf{q}_{ij}. \tag{40}$$

The position Jacobian blocks are

$$\mathbf{H}_{\mathbf{p}_i} = -\frac{\mathbf{R}_{z,i}^{\top}}{\Delta t_{ij}}, \qquad \mathbf{H}_{\mathbf{p}_j} = \frac{\mathbf{R}_{z,i}^{\top}}{\Delta t_{ij}}. \tag{41}$$

We derive the orientation block through the yaw anchoring of the local frame. Let $\mathbf{e}_z = [0, 0, 1]^{\top}$. A small perturbation of the initial clone yaw produces

$$\delta \left(\mathbf{R}_{z,i}^{\top} \mathbf{q}_{ij} \right) = \left[\mathbf{R}_{z,i}^{\top} \mathbf{q}_{ij} \right]_{\times} \mathbf{e}_z \delta\psi_i. \tag{42}$$

The yaw perturbation is related to the left attitude perturbation by

$$\delta\psi_i = \mathbf{j}_{\psi,i} \delta\boldsymbol{\theta}_i, \tag{43}$$

where $\mathbf{j}_{\psi,i} \in R^{1\times 3}$ is the yaw-extraction Jacobian evaluated at $\mathbf{R}_{B_i}^W$. For a yaw-pitch-roll parameterization with pitch ϑ_i and yaw ψ_i, it can be obtained that

$$\mathbf{j}_{\psi,i} = \begin{bmatrix} \cos\psi_i \tan\vartheta_i & \sin\psi_i \tan\vartheta_i & 1 \end{bmatrix}. \tag{44}$$

Thus, the initial orientation block is

$$\mathbf{H}_{\boldsymbol{\theta}_i} = \left[\hat{\mathbf{z}}_{ij} \right]_{\times} \mathbf{e}_z \mathbf{j}_{\psi,i}. \tag{45}$$

All other blocks are zero:

$$\mathbf{H}_{ij} = \begin{bmatrix} \cdots & \mathbf{H}_{\boldsymbol{\theta}_i} & \mathbf{H}_{\mathbf{p}_i} & \cdots & \mathbf{0}_{3\times 3} & \mathbf{H}_{\mathbf{p}_j} & \cdots \end{bmatrix}. \tag{46}$$

This sparse Jacobian makes the update efficient while preserving the historical correlations required by the segment-level observation.

Kalman update and marginalization. Given $\mathbf{r}_{ij}$ and $\mathbf{H}_{ij}$, the innovation covariance and Kalman gain are

$$\mathbf{S}_{ij} = \mathbf{H}_{ij}\boldsymbol{\Sigma}\mathbf{H}_{ij}^{\top} + \mathbf{N}_{ij}, \qquad \mathbf{K}_{ij} = \boldsymbol{\Sigma}\mathbf{H}_{ij}^{\top}\mathbf{S}_{ij}^{-1}. \tag{47}$$

The error-state correction is

$$\delta\mathcal{X} = \mathbf{K}_{ij}\mathbf{r}_{ij}. \tag{48}$$

The correction is injected into both cloned poses and the current state:

$$\begin{aligned} \mathbf{R}_{B_n}^{W} &\leftarrow \mathrm{Exp}(\delta\boldsymbol{\theta}_n)\mathbf{R}_{B_n}^{W}, \quad \mathbf{p}_n^{W} \leftarrow \mathbf{p}_n^{W} + \delta\mathbf{p}_n, \\ \mathbf{R}_{B}^{W} &\leftarrow \mathrm{Exp}(\delta\boldsymbol{\theta})\mathbf{R}_{B}^{W}, \quad \mathbf{v}^{W} \leftarrow \mathbf{v}^{W} + \delta\mathbf{v}, \\ \mathbf{p}^{W} &\leftarrow \mathbf{p}^{W} + \delta\mathbf{p}, \quad \mathbf{b}_g \leftarrow \mathbf{b}_g + \delta\mathbf{b}_g, \quad \mathbf{b}_a \leftarrow \mathbf{b}_a + \delta\mathbf{b}_a. \end{aligned} \tag{49}$$

The covariance is updated as

$$\boldsymbol{\Sigma} \leftarrow \boldsymbol{\Sigma} - \mathbf{K}_{ij}\mathbf{H}_{ij}\boldsymbol{\Sigma}. \tag{50}$$

The covariance is then symmetrized:

$$\boldsymbol{\Sigma} \leftarrow \frac{1}{2}\left(\boldsymbol{\Sigma} + \boldsymbol{\Sigma}^{\top}\right). \tag{51}$$

After each update, the oldest clone $i = j - m$ is marginalized by deleting its state entries and the corresponding covariance rows and columns, leaving the m most recent clones for the next update. Although the observation directly involves only the two endpoint clones, the Kalman correction also adjusts the intermediate clones and the current inertial state through their cross-covariances. This keeps the filter size bounded while retaining the historical correlations needed for subsequent velocity-consistency updates.

Discussion. The proposed filter is designed to bridge dense velocity prediction and probabilistic inertial state estimation. Dense predictions preserve fine-grained temporal motion cues during learning, while the segment-level average-velocity observation provides a robust and estimator-compatible constraint during inference. In addition, such a design is particularly suitable for heterogeneous carriers: instantaneous velocity estimates may be affected by carrier-specific vibration, contact, or aerodynamic disturbances, whereas their segment-level velocity consistency remains a stable cue for correcting inertial drift. As a result, the EKF acts as a history-aware integration layer that converts MosaicIMU's learned local motion priors into corrected inertial trajectories.

B Experimental Results

B.1 Multi-Carrier IMU Training Datasets

To evaluate multi-carrier generalization in neural inertial odometry across heterogeneous carriers, we construct a multi-carrier IMU benchmark from several publicly available datasets. As summarized in Table 5, the training datasets cover four representative carrier types, including vehicles, quadruped robots, pedestrians, and drones. To mitigate bias toward dominant motion categories, we standardize the training distribution by re-weighting samples from different carriers. This ensures that each carrier group contributes approximately the same proportion of training samples.

To avoid overlap leakage, all data splits are performed before sliding-window extraction. For each dataset, complete trajectories are first divided into training and held-out test trajectories with a 9:1 ratio. The held-out test trajectories are never used during training, validation, early stopping, hyperparameter tuning, or normalization-statistic estimation. For each training trajectory, the first 5% and the last 15% of data are reserved as validation segments, while the middle 80% is used for training. Sliding windows are generated separately after temporal segmentation. Windows crossing train/validation boundaries are discarded, and a one-window-length margin is removed around each split point to prevent raw IMU samples from being shared across training and validation windows. All raw IMU sequences are resampled to 200 Hz, and each valid segment is divided into sliding windows of 500 frames with a stride of 250 frames.

During training, we apply lightweight but physically consistent data augmentation. Specifically, zero-mean Gaussian white noise is added to the IMU measurements to improve robustness against sensor noise. In addition, we randomly rotate each IMU window around the gravity axis, together with its corresponding velocity labels, to augment heading variations while preserving the gravity direction and local inertial dynamics.

Table 5: **Multi-carrier IMU datasets used for training and evaluation.**

Carrier	Dataset	Duration	Environment
Vehicle	i2Nav-Robot [29]	4h	Indoors, Outdoors
	SubT-MRS [30]	3h	In/Outdoors, Underground
Quadruped	CEAR [31]	4h	Indoors, Outdoors
	Cerberus [32]	2h	Underground
Pedestrian	IDOL [33]	5.4h	Indoors
	RNIN-VIO [22]	5h	Indoors, Outdoors
Drone	BlackBird [34]	$<$1h	Indoors
	MILUV [35]	3.6h	Indoors

B.2 Details of Compared Baselines

To validate the generalization ability of MosaicIMU, we compare it with three representative open-source learning-based inertial odometry baselines: IMO [11], TLIO [6], and TartanIMU [13]. These methods cover carrier-specific neural inertial estimation, filter-based neural-inertial fusion, and a recent multi-carrier IMU pretrained model. For fair evaluation, IMO and TLIO are trained separately on the corresponding carrier data, while TartanIMU and MosaicIMU are trained on the same multi-carrier pretraining datasets. All four methods are evaluated on identical held-out sequences.

- **IMO** [11] is designed for autonomous drone racing. It combines an IMU-propagated EKF with a temporal convolutional network that predicts relative 3-DoF displacement from collective thrust and gyroscope measurements. The predicted displacement is then used as an EKF update. We include it as a representative drone-specific learned inertial odometry baseline. However, because IMO does not enforce rigid kinematic constraints, it can effectively adapt to various agents by simply training dedicated models on the respective carrier's data.
- **TLIO** [6] predicts local inertial displacement and uncertainty from short IMU windows, and incorporates the learned motion constraint into an error-state Kalman filter. Unlike cross-carrier models, TLIO learns a single motion prior and is not explicitly designed to compose heterogeneous carrier dynamics.
- **TartanIMU** [13] learns IMU representations from heterogeneous carriers and uses carrier-specific output heads for different platforms. It is the closest baseline to our model, but its fixed-head design requires explicit carrier assignment and does not support sample-adaptive expert composition. Since TartanIMU predicts velocities in its local body frame, we fuse its outputs using a standard velocity-observation EKF rather than the proposed history-aware EKF to obtain trajectories.

Excluded methods. We do not include some other inertial odometry methods whose assumptions are less compatible with our proposed model. For example, AI-IMU [7] introduces vehicle-specific motion assumptions such as non-holonomic constraints, which are effective for ground vehicles but difficult to generalize to pedestrians, quadruped robots, and drones. RoNIN [9] relies on transforming inertial predictions with an approximate rotation estimate, making it sensitive to the availability and quality of external attitude priors. Since our goal is to evaluate generalizable IMU-only velocity learning across heterogeneous carriers, we focus on the three open-source baselines above that can be adapted consistently to all evaluated platforms.

B.3 Evaluation Metrics

We compute all metrics using the estimated positions $\hat{\mathbf{p}}_t^W$ and ground-truth positions $\mathbf{p}_t^W$ in the world frame W.

Absolute Trajectory Error (ATE). ATE measures the root mean squared position error over the full trajectory:

$$\mathrm{ATE} = \sqrt{\frac{1}{M}\sum_{t=1}^{M} \|\hat{\mathbf{p}}_t^W - \mathbf{p}_t^W\|_2^2}, \tag{52}$$

where M is the number of evaluated timestamps.

Relative Trajectory Error over 10 seconds (RTE-10s). RTE-10s measures local drift by comparing estimated and ground-truth displacements over 10-second intervals:

$$\text{RTE-10s} = \sqrt{\frac{1}{M-\Delta}\sum_{t=1}^{M-\Delta} \left\|(\hat{\mathbf{p}}_{t+\Delta}^W - \hat{\mathbf{p}}_t^W) - (\mathbf{p}_{t+\Delta}^W - \mathbf{p}_t^W)\right\|_2^2}, \tag{53}$$

where Δ is the number of sampling intervals corresponding to 10 seconds.

Vertical trajectory error (z-Drift). z-Drift is ATE computed along the world-frame z-axis:

$$z\text{-Drift} = \sqrt{\frac{1}{M}\sum_{t=1}^{M} (\hat{p}_{t,z}^W - p_{t,z}^W)^2}. \tag{54}$$

All three metrics are reported in meters.

B.4 Ablation Studies of Pretrained MosaicIMU

Ablation on prediction in different frames. We further evaluate velocity prediction in different frames in Figure 5 and Table 6. The global-frame variant predicts velocity in the world frame W, while the body-frame variant uses the instantaneous body frame B_t. The proposed algorithm predicts velocity in the local gravity-aligned frame L_0, which keeps a stable local reference while decoupling the observation from global yaw.

The L_0 representation achieves the best overall performance across all evaluated carriers and metrics. As reported in Table 6, it reduces the average ATE from 8.00 m to 4.15 m compared with the B_t and W variants, and lowers the average RTE-10s from 1.93 m to 1.31 m. The improvement is particularly evident on vehicle and pedestrian sequences, where L_0 substantially reduces large trajectory deviations. Figure 5 further shows that the two alternative frames fail in different ways: global-frame prediction can couple velocity observations with attitude errors from the EKF, leading to accumulated trajectory drift, whereas body-frame prediction changes continuously with the rotation of the body, making dense velocity regression less stable. In contrast, the local gravity-aligned frame L_0 provides a fixed and gravity-consistent reference for each window, yielding more stable velocity learning and more reliable segment-level EKF updates without introducing spurious global-yaw constraints.

Ablation on expert design. Table 7 and Figure 6 report per-carrier results for different expert designs. As shown in Figure 7, the single-carrier expert baseline trains four independent single-expert networks, each using data from only one carrier. The mixture single expert uses one expert trained on all carriers without any carrier-specific branch. MosaicIMU uses the router to adaptively combine expert features for each input window.

As shown in Table 7, MosaicIMU achieves the best ATE and RTE-10s across all carriers. Compared with the single-carrier expert baseline, it reduces the average ATE from 5.47 m to 4.15 m

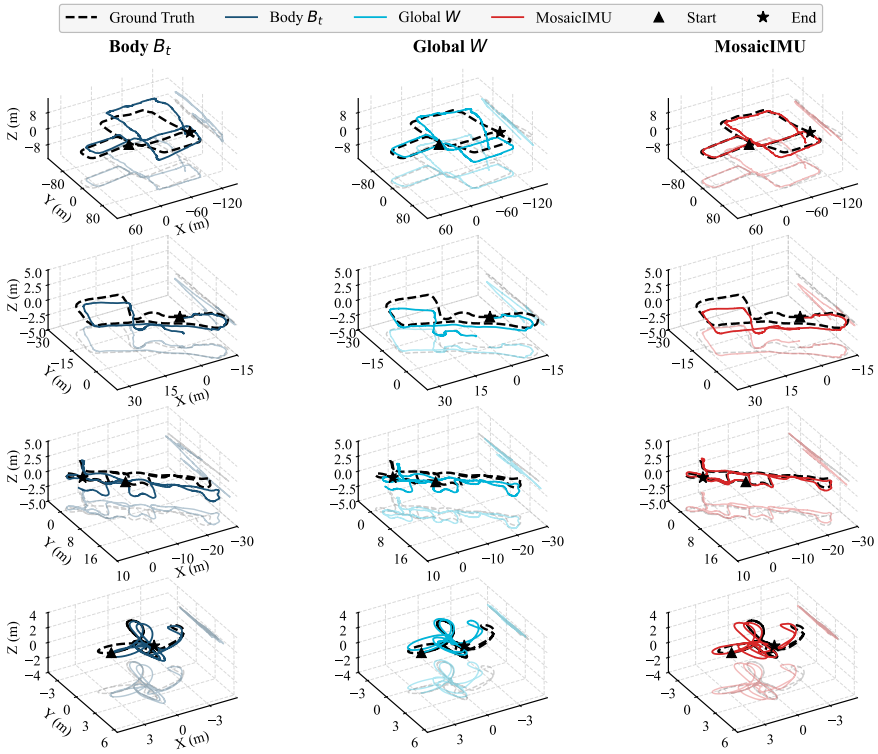


Figure 5: **Trajectory comparison under different velocity prediction frames.** From top to bottom, the four rows correspond to vehicle, quadruped robot, pedestrian, and drone sequences, respectively.

Table 6: **Generalization evaluation under different velocity prediction frames.** Unit: m.

Carrier	Body B_t			Global W			MosaicIMU L_0		
	ATE ↓	RTE-10s ↓	z-Drift ↓	ATE ↓	RTE-10s ↓	z-Drift ↓	ATE ↓	RTE-10s ↓	z-Drift ↓
Vehicle	14.64	3.26	3.68	26.76	3.95	1.91	**10.56** 28%↓	**2.17** 33%↓	**1.50** 21%↓
Quadruped	2.44	0.69	1.38	2.55	0.93	1.56	**2.01** 18%↓	**0.63** 9%↓	**0.86** 38%↓
Pedestrian	13.47	2.16	1.92	9.12	3.89	0.65	**2.85** 69%↓	**1.28** 41%↓	**0.35** 46%↓
Drone	1.44	1.62	0.92	3.17	2.18	0.25	**1.17** 19%↓	**1.14** 30%↓	**0.22** 12%↓
Average	8.00	1.93	1.98	10.40	2.74	1.09	**4.15** 48%↓	**1.31** 32%↓	**0.73** 33%↓

and the average RTE-10s from 2.09 m to 1.31 m, suggesting that isolated carrier-specific training limits cross-carrier knowledge transfer. The mixture single expert has a higher average ATE of 5.53 m, showing that a single shared expert is insufficient to capture heterogeneous inertial dynamics. By contrast, MosaicIMU combines shared representation learning with carrier-aware expert specialization. These results verify the importance of sample-adaptive feature-level composition for multi-carrier inertial learning.

Ablation on soft routing weights. Table 8 evaluates whether MosaicIMU benefits from soft expert composition rather than hard carrier-specific routing. The one-hot variant activates only the expert associated with the carrier label, which preserves carrier specialization but prevents information sharing across experts. In contrast, MosaicIMU uses the router weights to softly compose expert features for each input window.

As shown in Table 8, soft routing improves the average ATE from 4.71 m to 4.15 m and the average RTE-10s from 1.34 m to 1.31 m. The improvement is consistent in ATE across all carriers, indicating that the routed experts are not merely independent carrier-specific modules. Instead, the router keeps the dominant carrier expert while allowing useful complementary features from other experts to participate in the prediction. The overall results show that soft feature-level composition

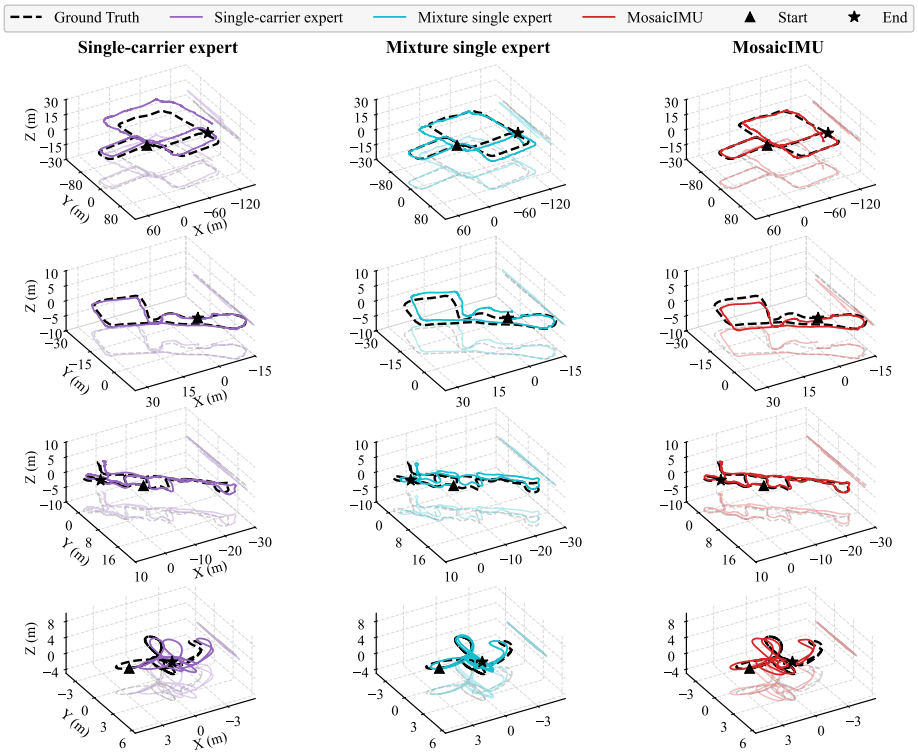


Figure 6: **Comparison of estimated trajectories from different expert structures.** From top to bottom, the four rows correspond to vehicle, quadruped robot, pedestrian, and drone sequences, respectively.

Table 7: **Generalization evaluation of different expert designs.** Unit: m.

Carrier	Single-carrier expert		Mixture single expert		MosaicIMU	
	ATE ↓	RTE-10s ↓	ATE ↓	RTE-10s ↓	ATE ↓	RTE-10s ↓
Vehicle	14.12	3.09	14.14	3.21	**10.56** 25%↓	**2.17** 30%↓
Quadruped	2.14	1.37	2.42	0.69	**2.01** 6%↓	**0.63** 9%↓
Pedestrian	3.99	2.45	4.25	1.82	**2.85** 29%↓	**1.28** 30%↓
Drone	1.61	1.45	1.31	1.22	**1.17** 11%↓	**1.14** 7%↓
Average	5.47	2.09	5.53	1.74	**4.15** 24%↓	**1.31** 25%↓

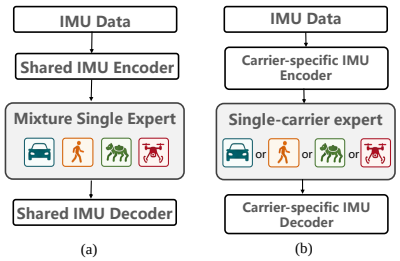


Figure 7: **Structural comparison of the baseline expert designs.** (a) Mixture single expert, and (b) Single-carrier expert. Note that both frameworks employ only a single expert network without a routing module.

provides a more flexible mechanism for heterogeneous inertial dynamics than a rigid one-hot expert assignment.

Table 8: **Ablation on soft routing weights.** We compare MosaicIMU with a one-hot routing variant that activates only the carrier-specific expert. Unit: m.

Carrier	**MosaicIMU** (one-hot)		**MosaicIMU**	
	ATE ↓	RTE-10s ↓	ATE ↓	RTE-10s ↓
Vehicle	12.40	2.31	**10.56** 15%↓	**2.17** 6%↓
Quadruped	2.36	0.68	**2.01** 15%↓	**0.63** 7%↓
Pedestrian	2.87	**1.20**	**2.85** 1%↓	1.28 7%↑
Drone	1.20	1.17	**1.17** 3%↓	**1.14** 3%↓
Average	4.71	1.34	**4.15** 12%↓	**1.31** 2%↓

B.5 Router Feature and Expert-Composition Analysis

We further analyze the behavior of the prototype router to better understand how MosaicIMU composes carrier-specific inertial features. As shown in Figure 8(a), the averaged routing matrix exhibits a clear diagonal-dominant structure. Each carrier assigns the largest routing weight to its corresponding expert, indicating that the learned prototypes preserve meaningful carrier semantics and that the experts develop carrier-specific inertial representations. Meanwhile, non-zero off-diagonal weights show that the router can also borrow complementary information from other carrier experts when forming the fused feature. This behavior is especially important for heterogeneous inertial data, where different platforms may share partial motion priors, vibration characteristics, or local velocity constraints despite having distinct overall dynamics.

Figure 8(b) visualizes the router feature embeddings together with the learned prototypes. Features from different carriers form distinguishable clusters, and the corresponding prototypes are located close to their associated feature regions, suggesting that the router learns a carrier-aware feature space. At the same time, the carrier clusters are not completely separated: local overlaps and neighboring regions can be observed among different carrier categories. This indicates that carrier dynamics are not purely discrete or mutually exclusive, but contain both carrier-specific and cross-carrier components. Therefore, imposing a rigid single-head prediction structure or a hard carrier-specific output partition may unnecessarily isolate useful shared inertial priors. In contrast, the proposed feature-level soft fusion preserves the dominant carrier-specific expert while allowing sample-adaptive feature sharing across experts, which provides a more flexible mechanism for multi-carrier inertial odometry.

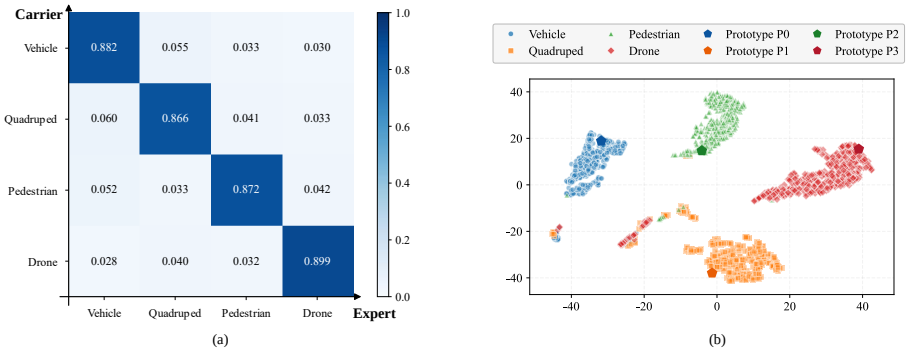


Figure 8: **Router output weight and feature visualization.** (a) Heatmap of routing-weight assignments between carriers and experts. (b) t-SNE visualization of carrier features and learned prototypes.

B.6 Ablation Studies of the Lightweight Offline Fine-tuning

To further examine the role of the router adapter in the fine-tuning stage, we remove both the router adapter and the prior loss used to supervise the routing distribution. The results are reported in Figure 9 and Table 9, while the corresponding results of MosaicIMU + New Expert are shown in Figure 3(a). The full method exhibits a much faster error convergence: it reaches the error level of the variant w/o adapter at 60 epochs using only about 20 epochs, and the error continues to decrease as the training proceeds. This demonstrates that carrier-prior guidance is effective for fine-tuning, as the corresponding carrier experts preserve motion knowledge that is beneficial for adapting to new data. However, due to the distribution mismatch between the features learned from the pretrained network and those of the new dataset, samples cannot be reliably assigned to the appropriate carrier experts. Introducing the router adapter alleviates this mismatch and enables the model to better exploit the prior knowledge stored in carrier-specific experts.

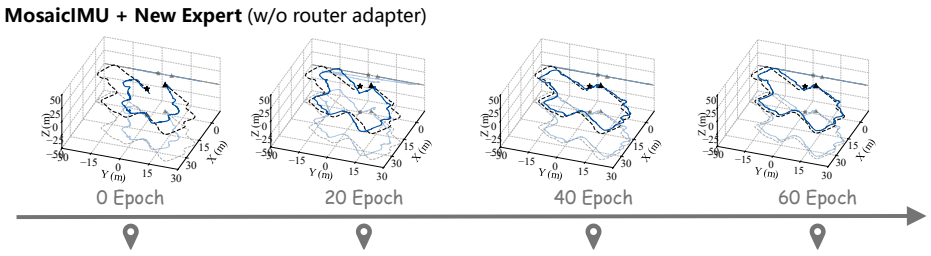


Figure 9: **Effect of the router adapter on fine-tuning convergence.** Trajectory estimates at different fine-tuning epochs without the router adapter.

Table 9: **Ablation of the router adapter in lightweight fine-tuning.** ATE and RTE-10s are reported at different epochs to compare convergence with and without the router adapter. Unit: m.

Method	0 Epoch		20 Epoch		40 Epoch		60 Epoch	
	ATE ↓	RTE-10s ↓	ATE ↓	RTE-10s ↓	ATE ↓	RTE-10s ↓	ATE ↓	RTE-10s ↓
New Expert w/o router adapter	9.85	5.36	8.88	2.44	3.20	1.55	2.64	1.12
New Expert	9.85	5.36	**2.60**	**1.32**	**2.22**	**0.88**	**1.49**	**0.72**

B.7 Details of Online Adaptation

We provide the implementation details of router-guided online adaptation. The experiment is conducted on a four-wheeled vehicle driving on outdoor campus roads. Figure 10 shows the on-board hardware configuration and the test environment, and Table 10 summarizes the main sensing and computing components. The trajectory in Figure 4(a) is used for online adaptation, while the trajectory in Figure 4(b) is a held-out sequence from the same target domain and is used only for evaluation. DLIO provides velocity supervision during online adaptation, but it is not used as an input to MosaicIMU during inference.

The online setting follows the lightweight adaptation strategy in Section 3.3: the pretrained base model is frozen, and only the router adapter and gated residual branch are updated. Since the target carrier is known to be a vehicle, we use the vehicle prior for router-guided adaptation. Incoming IMU windows are stored in an online buffer. At each update round, the calibrated router recomputes the mismatch score for the buffered windows, and only the Top-M high-mismatch samples are used to update the lightweight branch. In this experiment, we set M to 64. This strategy keeps the update focused on informative samples while retaining past observations for replay.

The online update is deployed on a Jetson AGX Orin. Each epoch takes less than 10 seconds, after which newly collected windows are added to the buffer, and the selection-update procedure is repeated. In Figure 4(c), the vehicle finishes the adaptation trajectory after approximately 30 epochs.

We then perform 10 additional epochs of buffer replay using the collected windows, which further stabilizes the adapted model.



(a) (b)

Figure 10: **Online adaptation platform and test environment.** (a) On-board sensing and computing setup. (b) Four-wheeled vehicle operating on outdoor campus roads.

Table 10: **Specifications of the online adaptation platform.** We summarize the main sensing, computing, and supervision components used in the online adaptation experiment.

Type	Model	Specification
LiDAR	Livox Mid-360	$\leq$2 cm, 1, @10 m
IMU	Xsens MTi-30	18°/h; 15 μg
Edge computing platform	Jetson AGX Orin	10.65 FP16 TFLOPS